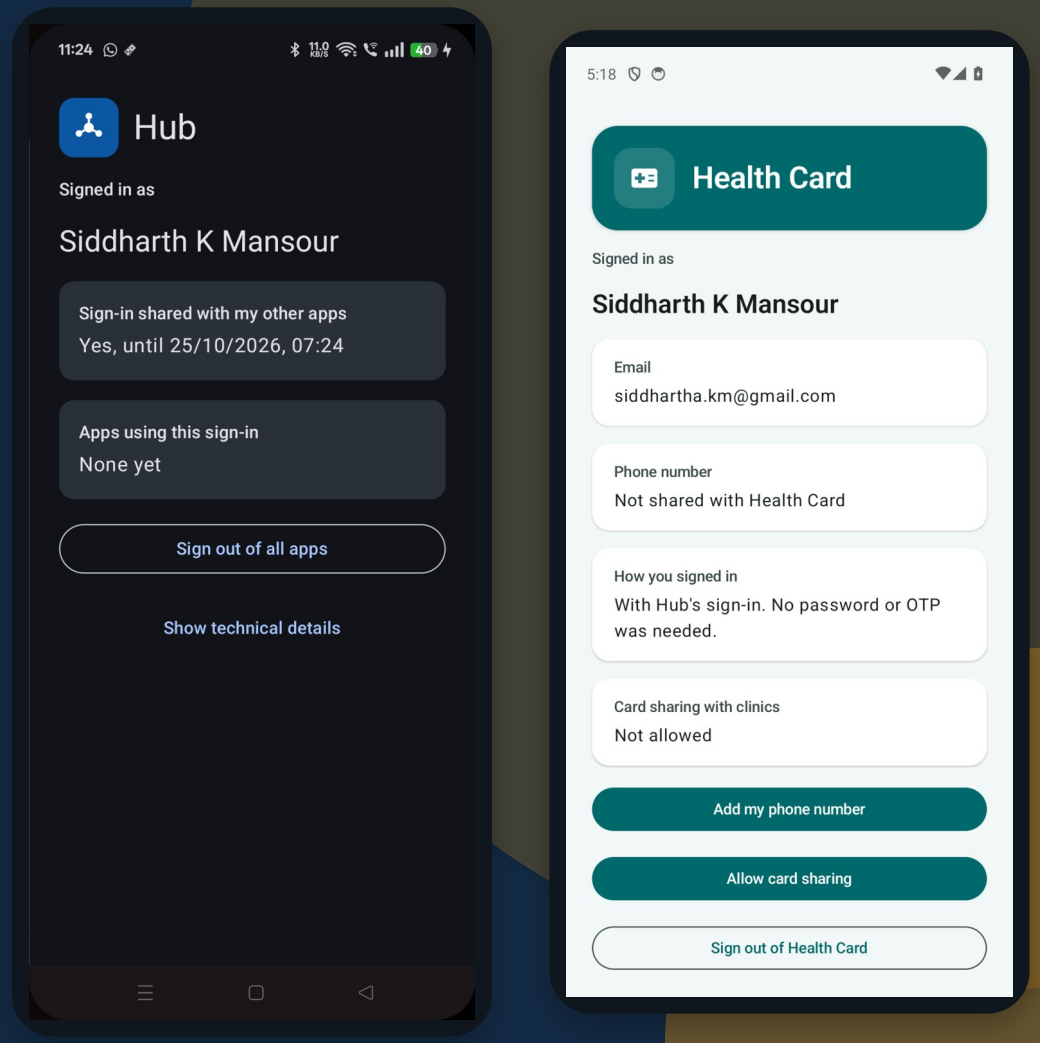
MOSIP DECODE 2026 · PROBLEM STATEMENT 03

Native SSO for mobile apps in eSignet Go

Sign in once with an OTP; the next app from the same publisher signs in with no browser and no second OTP



Team Badmosip

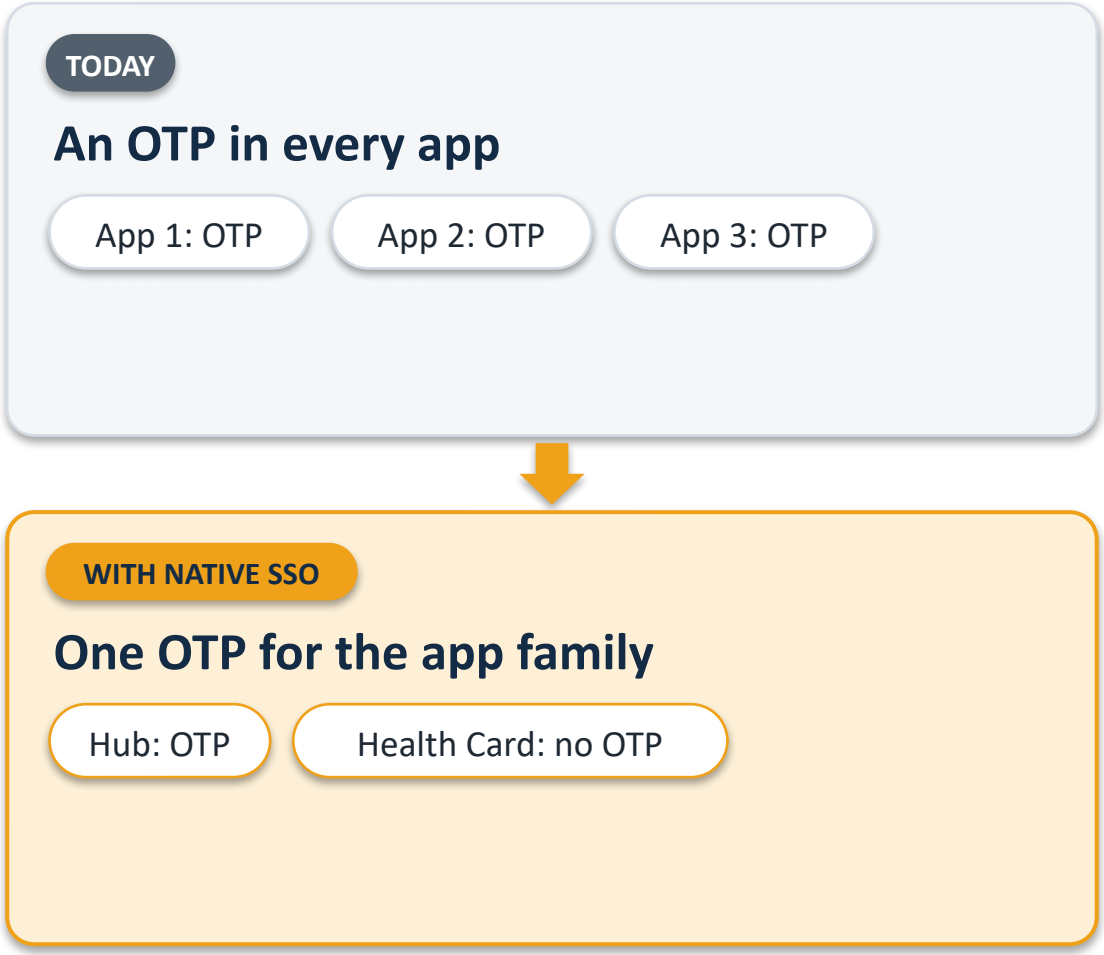
Kanav Bhardwaj · Ivan Bhargava · Nikunj Mahajan
Ishaan Sodhi · Harsh Modi

Every app asks for its own OTP

1. **Three apps, three logins**
   one agency, an OTP in each app

2. **Each OTP costs**
   an SMS, a delay, one more chance to be phished

3. **Native apps share no browser session**
   eSignet Go had no way to share a sign-in



TARGET AUDIENCE

One sign-in serves citizens, agencies and publishers



Citizens

using several apps from one publisher



Agencies running MOSIP eSignet

every extra OTP is one more SMS



Publishers of app families

no standard way to share a sign-in

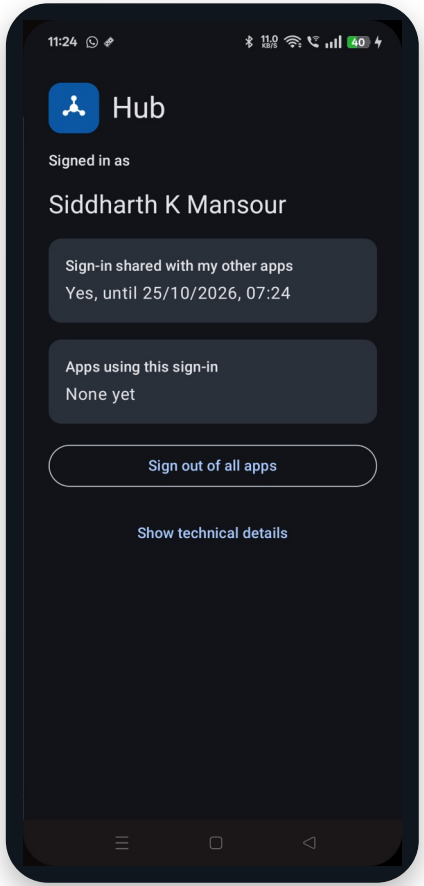
Sign in once, and the next app needs no OTP

1. **Hub signs in once**
   OTP login with the scope device_sso

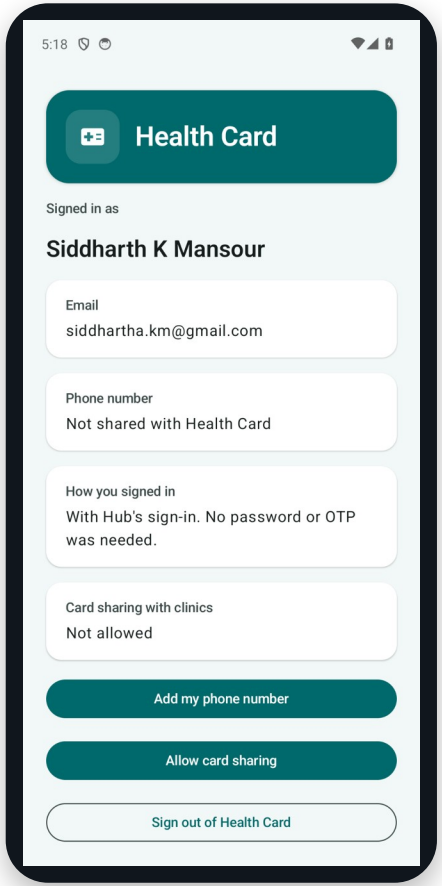
2. **eSignet returns a device secret**
   bound to Hub's DPoP key

3. **Health Card asks Hub**
   over a bound Android service

4. **Token exchange**
   Health Card gets its own tokens (RFC 8693)



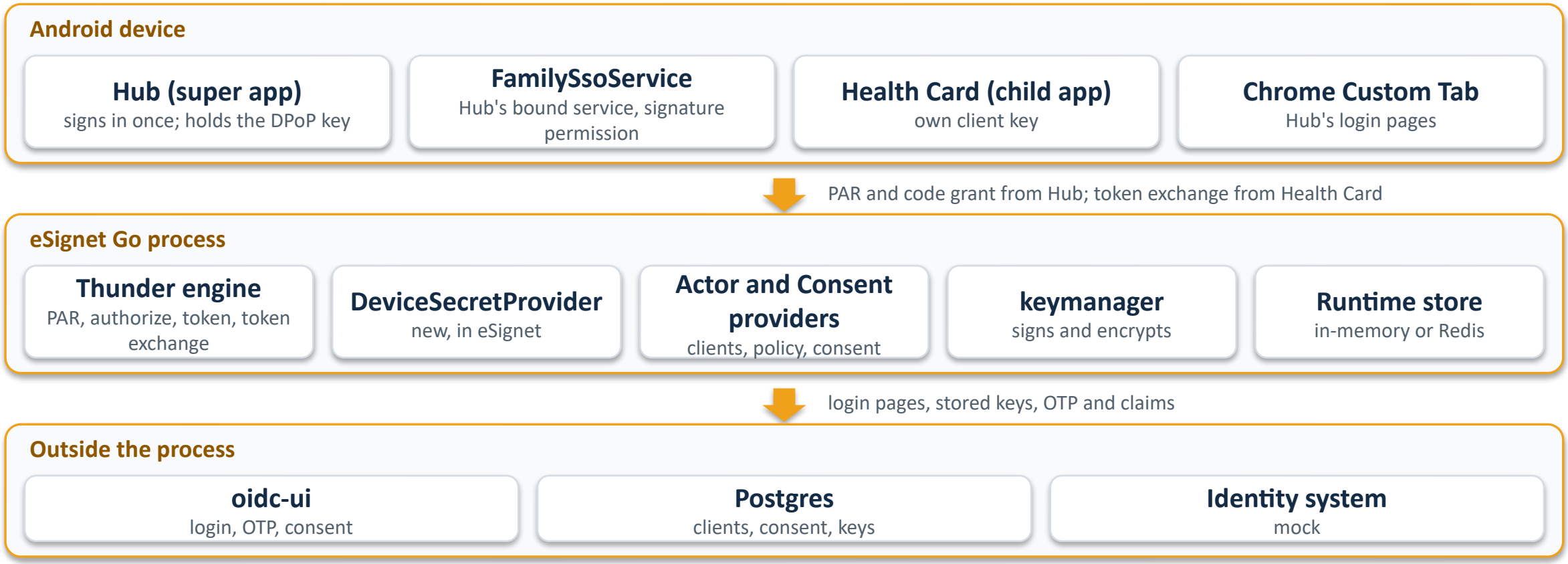
Hub signed in, on a real phone



Health Card, no login

ARCHITECTURE

Thunder runs the protocol; eSignet holds policy and keys



Hub signs every DPoP proof itself and never hands over its key; Health Card's tokens are bound to Hub's key.

TECHNOLOGY STACK

Go on the server, Kotlin on the phone, RFCs in between

01 Languages

- Go: Thunder ID engine and eSignet Go
- Kotlin: Hub and Health Card apps

02 Android

- Jetpack Compose screens
- Custom Tabs for the login page
- Keystore DPoP key, encrypted token store

03 MOSIP

- eSignet Go v2.0.0 on Thunder v1.0.1
- oidc-ui login and consent pages
- keymanager, mock identity system

04 Standards and infrastructure

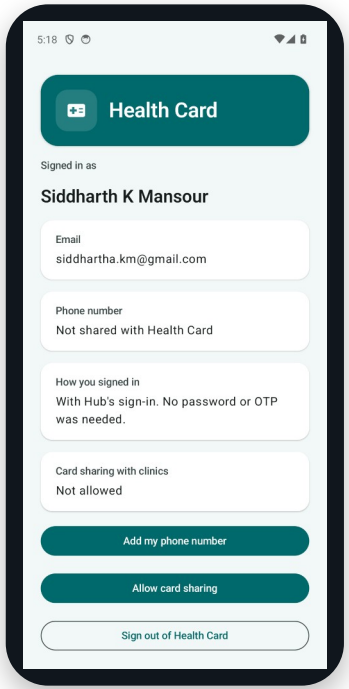
- PAR, PKCE, private_key_jwt, DPoP, RFC 8693
- Docker Compose, PostgreSQL, Redis
- newman, golangci-lint, govulncheck, GitHub Actions

DEMO

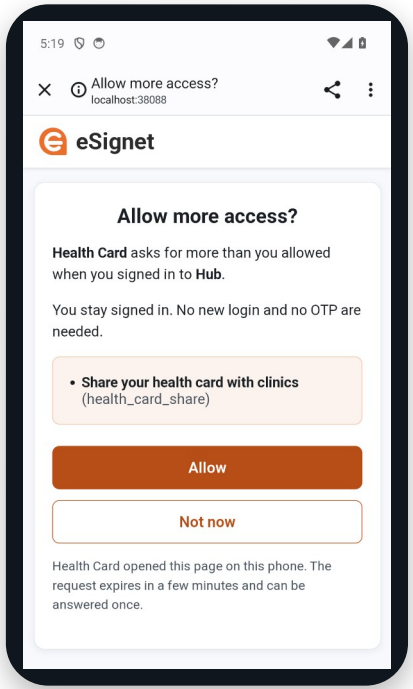
From OTP login to silent sign-in to sign-out of all apps

1. **Hub, sign in**
   OTP login; Hub shows "Yes, until <time>"
2. **Health Card, no login**
   silent sign-in through Hub
3. **Re-consent**
   one tap on eSignet's page
4. **Sign out of all apps**
   Health Card's tokens refused
5. **Hub's id_token expired**
   silent sign-in still works

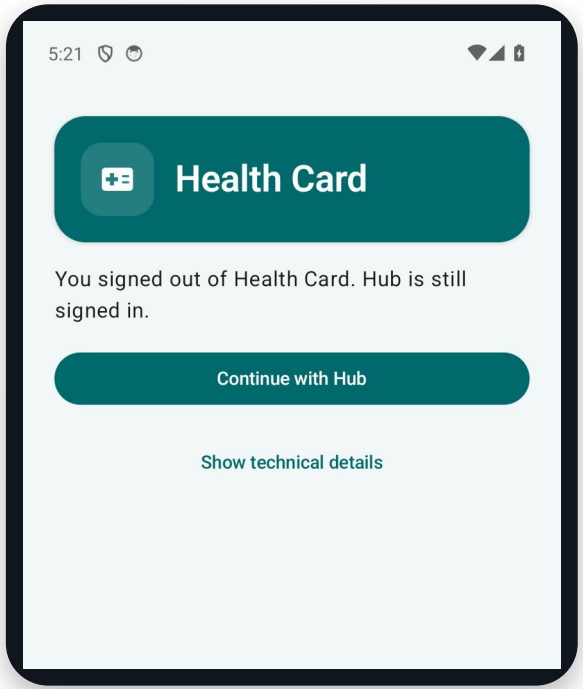
Demo video:
https://youtu.be/TqObNRZVifU



No login



Consent, no login



Signed out

EVIDENCE

31 live tests pass, and one real phone passes too

31 / 31

live end-to-end tests

in-memory store and Redis store

210

Android JVM tests, 0 failed

Hub, Health Card, shared libraries

1 phone

OnePlus CPH2569, Android 15

sign-in, silent sign-in, consent, sign-out, TalkBack, large font: all passed

Also measured: 2467 + 1613 server unit tests, 0 failed (Thunder and eSignet) · 6 / 6 emulator UI tests (plus the family sign-in steps) · 98.6 % changed-line coverage, Thunder (91.7 % in eSignet) · Server and emulator runs 2026-10-25; real-phone run 2026-10-24. Postman: 40 requests, 146 assertions, 0 failed under newman.

REQUIREMENTS COVERED

All ten mandatory tasks and the bonus are built and tested

Mandatory tasks 1 to 5

- **T1 device_sso scope**
  test-report.md R01
- **T2 Token endpoint check**
  test-report.md R02
- **T3 Device Secret Provider**
  test-report.md R03
- **T4 Generate, validate secret**
  test-report.md R04, R09
- **T5 Called from token endpoint**
  test-report.md R05, R08

Mandatory tasks 6 to 10

- **T6 Tokens and secret back**
  test-report.md R06
- **T7 App-to-app retrieval**
  test-report.md A04a, A04b
- **T8 Token exchange call**
  test-report.md R23
- **T9 Token Exchange grant**
  test-report.md R10 to R14
- **T10 Scope rules**
  test-report.md R15 to R17

Bonus and deliverables

- **B1 Re-consent screen**
  test-report.md R28
- **Native apps, full flow**
  android/README.md
- **Postman collection**
  esignet/postman-collection/
- **API and design documents**
  docs/api.md, docs/design.md

Tick: done, with the file shown. Paths are under docs/ unless they start with android/ or esignet/. Source: docs/test-report.md, docs/submission-checklist.md.

IMPACT

Zero OTPs for the second app

Who benefits



Citizens
one OTP for the whole app family



Agencies
fewer SMS messages per user



App publishers
a reusable DeviceSecretProvider

What changes, counted

0

OTPs for the second app

no browser window; shown on a real phone

1 tap

to grant more scope

no new login (bonus B1)

1 tap

signs out every app

tokens refused afterwards (R27)

12

threats, each with a test

STRIDE per component

A device secret that reveals nothing, and a key that never leaves Hub

1. **A device secret that reveals nothing**
   signed then encrypted JWT, bound to Hub's key

2. **Hub signs, never shares its key**
   proofs for allow-listed endpoints

3. **Consent without a second login**
   the user's own browser login approves; the app cannot

4. **Reusable provider interface**
   any eSignet deployment can implement it

1. **Health Card binds to Hub's service**
   Hub checks the caller's signing certificate
2. **Hub hands over the sign-in**
   its id_token and the device secret
3. **Hub signs a DPoP proof**
   for the token endpoint; its key never leaves Hub
4. **Token exchange at eSignet**
   id_token and device secret, proof signed by Hub's key
5. **eSignet checks every binding**
   device secret, ds_hash and sid, Hub's key, scope
6. **Health Card gets its own tokens**
   no OTP, no browser

Health Card sign-in through Hub, step by step

Scope and next steps

Scope

- Silent sign-in lasts as long as Hub's device secret, 8 hours by default (L10)
- New user data (the phone number) asks for the OTP again (L02)
- After sign-out, a server that checks the signature alone accepts a token until it expires (L04)

Runs still to do

- Software-backed Keystore on the emulator (L01)
- Other phones and Android versions: one phone used (L09)
- Postman desktop app run (L07)

ROADMAP

Next: offer the changes upstream, then attest the device



Offer the changes upstream

to Thunder and eSignet (docs/upstream-prs/)

Device attestation

before a device secret is issued

Several replicas

the Redis store already passes the live suite

iOS and more phones

other Android versions and a phone with StrongBox (L06, L09)

TEAM

Team Badmosip

KB

Kanav Bhardwaj

IB

Ivan Bhargava

NM

Nikunj Mahajan

IS

Ishaan Sodhi

HM

Harsh Modi

Repository

https://github.com/Paradox-73/Enable_Native_SSO_in_eSignet_Go_for_Mobile_Apps

Public on GitHub, with the code, the test reports and the screenshots behind every number in this deck

TRY IT

Run it yourself

```
$  bash scripts/with-stack.sh bash e2e/run.sh
```

You need Docker (Linux containers) and bash 4.4 or newer; Go runs in a container. Clone with core.longpaths=true on Windows. The apps and the emulator demo also need JDK 17 or newer and the Android SDK.

You see The stack starts, 31 live tests run, the stack stops, and the last line reads: e2e: 31 passed, 0 failed.

Scan for the repository

Repository **https://github.com/Paradox-73/Enable_Native_SSO_in_eSignet_Go_for_Mobile_Apps**